Understanding Interactions in Regression Models

Categorical × numerical and numerical × numerical interactions, marginal vs. conditional effects, and common pitfalls in ecological data analysis

Allen Bush-Beaupré

2026-10-05

Welcome

This workshop is adapted from a three-part tutorial series on interactions in regression models:

- Part 1: Categorical × numerical interactions
- Part 2: Numerical × numerical interactions
- Part 3: Generalized linear models and link functions

Audience: graduate students and researchers in ecology who fit GL(M)Ms and want to interpret interaction terms correctly.

Format: short lecture blocks, and practice exercises.

Learning objectives

By the end of this workshop you should be able to:

1. Explain what an interaction term means for a categorical × numerical model and for a numerical × numerical model
2. Distinguish **conditional** effects from **marginal** effects, and choose the right one for a research question
3. Use `marginaleffects` (`predictions()`, `slopes()`, `avg_predictions()`, `avg_slopes()`, `hypotheses()`) to extract and test the quantities you actually care about

Learning objectives (continued)

4. Visualize interaction effects (slope-by-moderator plots, five-number-summary predictions, response surfaces) and know their trade-offs
5. Recognize that a non-linear link function (logit, log) creates an **implicit interaction** even when the model formula has none, and convert a raw GLM coefficient to a probability-scale or count-scale effect
6. Interpret a hurdle/zero-inflated model's two components separately and combined, and recognize that two-way interactions are symmetric

Actual learning objective

Model summary outputs rarely answer a research question and definitely do not communicate the results in a comprehensible way.

Setup

```
 1 library(tidyverse)        # Data manipulation and plotting
 2 library(glmmTMB)          # Generalized linear (mixed) models
 3 library(marginaleffects) # Compute marginal/conditional effects
 4 library(patchwork)        # Combine multiple plots
 5 library(GGally)           # Pairwise plots
 6 library(distributional)  # Simulate mixture/gamma/normal distributio
 7 library(ggdist)           # stat_slab() for predicted-distribution pl
 8 library(tinytable)        # tt(): simple, nice-looking results tables
 9 library(performance)     # r2(), r2_tjur(): model R-squared measures
10
11 # Set ggplot2 theme
12 black_theme <- theme(
13   axis.line = element_line(linewidth = 2, lineend = "round", color =
14   panel.grid = element_blank(),
15   panel.background = element_rect(fill = "black", color = NA),
16   axis.ticks = element_blank(),
```

Coding style & functions used throughout

A few conventions and workhorse functions you'll see on repeat:

- **Native pipe `|>`**: `data |> mutate(...) |> ggplot(...)` reads left-to-right, "and then"
- **`tibble()` + `rnorm()`/`rbinom()`/`rpois()`/`runif()`**: how we simulate data with a *known* true effect, so we can check whether our model recovers it
- **`glmmTMB(y ~ x1 * x2, data = ..., family = ...)`**: our model-fitting workhorse; `*` expands to `x1 + x2 + x1:x2`
- **`marginaleffects` verbs**:
  - `predictions()` / `avg_predictions()` — model-implied values of Y
  - `slopes()` / `avg_slopes()` — model-implied *effects* (derivatives) of a predictor on Y
  - `datagrid()` — build a covariate grid to evaluate predictions/slopes at (e.g., `fivenum`, specific values, `mean`)
  - `by =` — keep results split by a grouping variable instead of averaging over it
  - `hypotheses()` — test linear combinations of estimates (e.g., "is slope A different from slope B?")
- **`set.seed()`**: called once per script so simulated data (and thus every number you see) is reproducible

Part 1 — Categorical × Numerical Interactions

What is an interaction?

An interaction means **the effect of one predictor depends on the value of another**.

- No interaction: snow depth affects weight the same way for every sheep
- Interaction: the effect of snow depth on weight *differs by sex*

For a categorical moderator (e.g., sex) and a continuous predictor (e.g., snow depth): the interaction means the **slope** differs between categories.

Our focus today isn't the algebra — it's using `marginaleffects` to find, test, and plot "it depends."

Conditional vs. marginal effects

- **Conditional effects**: the effect of a predictor *at specific values* of other predictors (e.g., the slope of snow depth *for males*)
- **Marginal effects**: the effect *averaged across* the distribution of other predictors (e.g., the *average* slope of snow depth across the whole population)

Both are legitimate; they answer different questions. Interactions are exactly where the distinction starts to matter.

Case study: Bighorn sheep

Ram Mountain, Alberta. Question: how does winter snow depth relate to spring mass in Bighorn sheep, and does it differ by sex? Males and females segregate in winter — females use lower, less snowy habitat.

```
n_individuals_population <- 1000
sexes <- c("male", "female")
male_intercept <- 220; female_intercept <- 180
sd_weight <- 15
male_loss <- -0.8; female_loss <- -2.5

bighorn_population <- tibble(
  sex = rep(sexes, n_individuals_population / length(sexes)),
  snow_depth = runif(n_individuals_population,
    min = ifelse(sex == "male", 35, 0),
    max = ifelse(sex == "male", 70, 35)
  ),
  weight = rnorm(n_individuals_population,
    mean = ifelse(sex == "male",
      male_intercept + male_loss * snow_depth,
      female_intercept + female_loss * snow_depth),
```

Built-in features: different intercepts, different slopes, **and** different snow-depth distributions by sex.

Simpson’s Paradox

Pooling males and females together and fitting a single trend line can hide — or even reverse — the true relationship.

Simpson's Paradox

- Code

Pooled vs. within-sex

Pooled: snow depth looks **positively** associated with weight — even though both sexes lose weight with more snow.

Why? Sex is a confounder

- Within each sex: more snow → lower weight (true negative relationship)
- Between sexes: males see more snow **and** are heavier (confounded)
- Pooled data compares heavy, high-snow males to light, low-snow females → spurious positive trend

Within-sex trends

- Code

Exercise 1

Using `bighorn_population`:

1. Fit `weight ~ sex * snow_depth` with `glmmTMB()`. Call it `mod_mass_true`.
2. From the summary, identify which coefficient represents the **male** slope of snow depth. (Hint: it's not printed directly — you have to add two rows together.)
3. In one sentence, explain why a naive `weight ~ snow_depth` model (ignoring sex) would mislead a manager trying to understand the effect of a harsh winter. Can you think of an example of such situation in your own field or research?

(~5 minutes — discuss with a neighbor)

Sampling from the population

In practice we can't measure everyone. Two sampling scenarios:

- **Scenario 1**: balanced trap, 50 males / 50 females
- **Scenario 2**: biased trap, 80 males / 20 females

```
n_individuals <- 100
bighorn_mass_1 <- bighorn_population |>
  group_by(sex) |>
  slice_sample(n = n_individuals / 2, replace = FALSE) |>
  ungroup()

mod_mass_1 <- glmmTMB(weight ~ sex * snow_depth,
                      data = bighorn_mass_1, family = gaussian())
```

Model coefficients

- Code

Reading the coefficients

- **Intercept**: female weight at snow_depth = 0
- **sexmale**: male − female intercept difference
- **snow_depth**: female slope
- **sexmale:snow_depth**: male − female slope difference

Useful for hypothesis tests, not directly interpretable for "what's the effect for males?" — that's what `marginaleffects` is for.

Q1: Do sexes differ in spring mass?

- Code

This is a **marginal prediction**: the model-implied mean for each sex, averaged over the observed snow-depth distribution in the sample.

Testing the sex difference

- Code

Q2: Does snow depth affect mass equally by sex?

▶ Code

Testing the slope difference

- Code

Q3: What's the population-average slope?

`avg_slopes()` computes the slope for every observation, then averages:

▶ Code

```
1 mean(c(male_loss, female_loss))  # true simulated average
```

Marginal vs conditional, visualized

▶ Code

Scenario 2: biased sampling (80% male)

- Code

Reading the biased-sample result

Conditional (by-sex) slopes stay close to truth. But the **marginal** slope is biased toward the overrepresented, weaker-effect sex (males).

Correcting for known sampling bias

If we *know* the true population sex ratio (50:50), we can average over a **balanced** `newdata` grid instead of our biased sample.

```
newdata_balanced <- bighorn_mass_2 |>
  group_by(sex) |>
  reframe(snow_depth = seq(min(snow_depth), max(snow_depth), length.
  ungroup()
```

Exercise 2

Using the model that was fit to the entire population and the one with a biased sample, use the avg_predictions() function along with a balanced and unbalanced prediction grid to create a plot comparing the marginal predictions between the true population, biased population predictions and corrected population predictions. There should be three trend lines.

(~15 minutes)

Comparing biased and unbiased prediction grids

- Code

Part 1 takeaways

1. Interactions → conditional effects that depend on other variables
2. Simpson's Paradox is a real risk when pooling across confounded groups
3. Marginal ≠ conditional; both are valid but answer different questions
4. Raw model coefficients rarely equal your research quantity of interest — use `marginaleffects`
5. Marginal effects inherit your **sample's** covariate distribution unless you override it with `newdata`

Part 2 — Numerical × Numerical Interactions

Two numerical predictors

With two continuous predictors, an interaction means: **the slope of one predictor changes as the other predictor changes** — not a separate slope per group, but a sliding one.

A raw coefficient is only “the slope when the other predictor equals zero” — meaningful only if zero is a meaningful value. This motivates **centering** predictors on their mean, so the intercept and main-effect coefficients describe *average* conditions.

Case study: petal length

Simulated flowers; petal length as a function of temperature and rain, with an interaction.

Predictors are **mean-centered**: the intercept becomes the predicted response at *average* conditions rather than at (biologically meaningless) zero temperature/rain.

Simulating the data

```r
n_flowers <- 200
temperature <- rnorm(n_flowers, mean = 10, sd = 15)
rain <- rgamma(n_flowers, shape = 2)
avg_petal_length <- 25.6; sd_petal_length <- 4.5
temperature_effect <- 0.4; rain_effect <- 0.9; interaction_effect <-

flower_measurements_1 <- tibble(
  temp_centered = temperature - mean(temperature),
  rain_centered = rain - mean(rain),
  petal_length = rnorm(n_flowers,
    mean = avg_petal_length +
      temperature_effect * temp_centered +
      rain_effect * rain_centered +
      interaction_effect * temp_centered * rain_centered,
    sd = sd_petal_length)
)
```

Fit the model

- Code

Every term has a small p-value — but what do these numbers *mean* for our research question? Time to unpack them.

Slope of one predictor at a single value of the other

▶ Code

This matches the `temp_centered` coefficient directly — because rain is centered, “at rain = 0” means “at average rain.”

Slope across the *range* of the moderator

- Code

Reading the slope-by-moderator plot

The interaction coefficient is exactly the slope of this line — how much the effect of rain *changes* per unit of temperature.

Visualizing predictions at chosen moderator values

Common choice: the five-number summary of the moderator.

▶ Code

Response surfaces — use with caution

▶ Code

Response surfaces: a caution

Contour bins are arbitrary and hide uncertainty — good for exploration, risky for publication-quality inference. Prefer line-and-ribbon plots when you can.

Exercise 3

Using `mod_flower_1`:

1. Compute the slope of **temperature** across the observed range of **rain** (mirror the rain-across-temperature plot above).
2. At what value of rain does the temperature effect become non-significant (CI crosses zero), if at all?
3. Rewrite the research question this model could answer as a single sentence, identifying which variable is the *exposure* and which is the *moderator*.

(~10 minutes)

Interactions beyond the mean: distributional regression

Interactions don’t have to act only on the average response. `glmmTMB`’s `dispformula` lets predictors affect the **standard deviation** too — useful when variance itself is ecologically meaningful (e.g., reduced trait variance under stress).

Simulating heteroscedastic data

```
log_sd_intercept <- log(sd_petal_length)
log_sd_temp_slope <- 0.03

flower_measurements_2 <- tibble(
  temp_centered = temperature - mean(temperature),
  rain_centered = rain - mean(rain),
  petal_length = rnorm(n_flowers,
    mean = avg_petal_length + temperature_effect * temp_centered,
    sd = exp(log_sd_intercept + log_sd_temp_slope * temp_centered))
)
```

The spread around the trend increases with temperature

▶ Code

Specifying the model

```
mod_flower_2 <- glmmTMB(petal_length ~ temp_centered,
                        dispformula = ~ temp_centered,
                        data = flower_measurements_2)
```

Checking the dispersion model

- Code

Reading the dispersion-model plot

`type = "disp"` retrieves predictions/slopes for the dispersion sub-model, exactly as `type = "conditional"` does for the mean.

Simulating and fitting interactions on the standard deviation

```
log_sd_rain_slope <- 0.02
log_sd_interaction_effect <- 0.005

tibble(temp_centered = temperature - mean(temperature),
       rain_centered = rain - mean(rain),
       petal_length = rnorm(
                        n = n_flowers,
                        mean = avg_petal_length +
                          temperature_effect * temp_centered +
                          rain_effect * rain_centered,
                        sd = exp(log_sd_intercept +
                           log_sd_temp_slope * temp_centered +
                           log_sd_rain_slope * rain_centered +
                           log_sd_interaction_effect* temp_centered

mod_flower_measurements_3 <- glmmTMB(petal_length ~ temp_centered +
```

Interpretation

We can interpret the interactive effects of rain and temperature on the standard deviation of petal length in the same way that we would for the average. Examples are shown in the Numerical x Numerical Interactions tutorial online. I want to display another way to communicate the results. Knowing that the two parameters of the Normal distribution act together to generate the observations, we can predict new distributions of observations for different predictor values.

Predicting new observations

- Code

Part 2 takeaways

Part 3 — Link Functions Create Implicit Interactions

A different kind of interaction

So far, every interaction we've seen was written explicitly into the model formula (`X1 * X2`).

Non-linear **link functions** (logit, log, …) introduce a *second*, implicit source of interaction: even a model with **no interaction term at all** can show predictors whose response-scale effect depends on the values of other predictors.

This happens because the link function bends a straight line (on the *link* scale) into a curve (on the *response* scale) — and how much a fixed push on the link scale moves the response depends on where you are on that curve.

Case study: species occurrence

Simulated presence/absence of a species across 200 plots, as a function of tree cover, rain, and temperature — **no interaction terms**, fit with a binomial GLM (logit link):

```
n_plots <- 200
tree_cover <- dist_mixture(dist_beta(4, 0.8), dist_beta(0.8, 4), wei
avg_yearly_rain <- dist_gamma(shape = 3, rate = 0.7)
avg_yearly_temp <- dist_normal(mu = 20, sd = 10)

occurence_intercept <- 0.047
occurence_tree_effect <- 0.45
occurence_rain_effect <- 0.02
occurence_temp_effect <- 0.03

occ_data <- tibble(
  tree_cov = generate(tree_cover, n_plots),
  rain = generate(avg_yearly_rain, n_plots),
  temp = generate(avg_yearly_temp, n_plots)
) |>
```

```
16    unnest(everything()) |>
```

Fit the model

- Code

We simulated the effect of tree cover to be **0.45** on the probability scale, but the model reports **~3.76**. Did the model get it wrong?

No — it’s reporting a different scale

The logit-link GLM draws a straight line through the **log-odds** of occurrence, then bends that line into an S-shape to get back to a probability.

The logit link, visualized

▶ Code

Reading the S-curve

Near the middle (p ≈ 0.5) the curve is steep: a small log-odds push moves probability a lot. Near the ends (p ≈ 0 or 1) the curve flattens: the same push barely moves probability.

No manual derivative needed — `marginaleffects` converts to the scale we want.

`marginaleffects` does this for us

- Code

`slopes()` in this case asks: “at these covariate values, how much does *predicted probability* change for a small increase in tree cover?” — a probability-scale answer, directly comparable to how we simulated the data.

The implicit interaction, visualized

▶ Code

Why the slope moves around

Because the amount of “bending” depends on where you are on the S-curve, tree cover’s probability-scale effect depends on rain and temp too — even though we never told the model to include that interaction.

A note on measurement scale (Spake et al. 2023)

Whether an interaction "exists" can depend on which scale you look at: **additive** (probability, or log-odds as a straight difference) vs. **multiplicative** (ratios, or the log scale itself).

`mod_1` has *no* interaction term, yet:

- on the **log-odds** scale: no interaction, by construction
- on the **probability** scale: an interaction appears, purely from the S-curve's shape

Getting the scale question wrong is one way real interactions get mis-detected or mis-sized (a "Type D" error in Spake et al.'s

terminology).

Response vs. link scale, across each moderator

▶ Code

Reading these panels

- **Link (log-odds) scale**: the slope of `tree_cov` is flat (~1.98) across both moderators — exactly as it should be, since `mod_1`'s formula has no interaction term at all.
- **Response (probability) scale**: the slope visibly curves with temp (and more weakly with rain) — the implicit interaction made concrete, purely from the logit link's curvature.
- With three predictors, a focal variable's slope can no longer be shown across "the" moderator in one 2D plot — each panel above holds the *other* remaining predictor fixed at its mean. Alternatives: facet by the third variable, use small multiples at representative (e.g. five-number-summary) values, or report a single `avg_slopes()` that integrates over the observed distribution instead of fixing it.

The “overall” effect of tree cover

▶ Code

`avg_predictions()`/`avg_slopes()` give one number (or curve) per focal value, integrating over rain and temp as observed in the sample — a marginal, “average effect” alternative to picking a single moderator value to condition on.

Exercise 4

Using `mod_1`:

1. Plot the predicted trend of occurence as a function of tree cover for the unique combinations of the five number summary of rain and temperature. Hint: use the predictions() function.
2. Plot the prediction for the average effect of temperature.

(~20 minutes)

Exercise 4: answer (task 1)

▶ Code

Exercise 4: answer (task 2)

▶ Code

Reviewer 2 wants a table

- Code

Case study: species abundance

The same phenomenon shows up with **count** data. Abundance (when present) as a function of the same three predictors, fit with a Poisson GLM (log link), again **no interaction terms**:

```
abundance_intercept <- exp(1)
abundance_tree_effect <- 4.5
abundance_rain_effect <- 0.1
abundance_temp_effect <- 0.1

abund_data <- tibble(
  tree_cov = generate(tree_cover, n_plots),
  rain = generate(avg_yearly_rain, n_plots),
  temp = generate(avg_yearly_temp, n_plots)
) |>
  unnest(everything()) |>
  mutate(abundance = rpois(n_plots, lambda =
    abundance_intercept +
    abundance_tree_effect * tree_cov +
    abundance_rain_effect * rain +
```

```
16      abundance_temp_effect * temp))
```

The log link bends a line into a curve

▶ Code

From log scale to counts

The log-scale `tree_cov` coefficient is **~0.73**, not 4.5 — the same phenomenon as the logit link, just a different curve.

Two ways to report a log-link coefficient

- **Rate ratio**: a *constant* multiplicative effect, valid everywhere — “each unit increase in tree cover multiplies expected abundance by this factor, regardless of rain or temp”
- **Local slope**: an *additive*, count-scale effect (“this many more individuals”), but only valid near the specific covariate values used

Computing the rate ratio

▶ Code

Both describe the same underlying model — pick the one that matches your audience (percent change vs. a concrete headcount).

Exercise 5

Using `mod_2`:

1. Compute the rate ratio and the local count-scale slope for `temp`, holding `rain` at its minimum (a “drought” scenario) and `tree_cov` at its mean.
2. Repeat holding `tree_cov` at a low vs. a high value (e.g., the 10th and 90th percentiles) — does the rate ratio change? Does the local slope?
3. Explain, in your own words, why one of these two summaries stays constant across `tree_cov` and the other doesn’t.

Exercise 5: answer (task 1)

- Code

Exercise 5: answer (task 2)

- Code

The rate ratio stays essentially constant across tree cover (it's a fixed multiplicative effect,

$$e^{\beta}$$

, independent of covariate values), while the count-scale slope grows substantially from low to high tree cover, since it's rescaled by the much larger expected count

Case study: occurrence *and* abundance together

In reality, a species first has to be **present** before we can measure its abundance. A **hurdle model** combines two sub-models fit jointly:

- A **binomial** (logit-link) component: does a plot clear the "hurdle" of zero? (occurrence)
- A **truncated count** component: given it's non-zero, how many individuals? (abundance when present)

Two ways to interpret: each part **separately** (what drives occurrence vs. abundance), or **combined** on the population

scale (expected abundance across *all* plots = P(occurrence) × E[abundance | present]).

A simple hurdle example

```r
# Simple hurdle simulation: abundance_pres = counts if present; occu
# abundance_pop = observed count in every plot (zero wherever the sp
tibble(
  tree_cov = generate(tree_cover, n_plots)
) |>
  unnest(everything()) |>
  mutate(abundance_pres = rpois(n_plots, lambda =
                        abundance_intercept +
                        abundance_tree_effect * tree_cov
                        ),
         occurence = rbinom(n_plots, size = 1,
                        prob = occurence_intercept +
                               occurence_tree_effect * tree_cov),
         abundance_pop = abundance_pres * occurence) -> simple_pop_d

# Hurdle model: main formula is the (zero-truncated Poisson) abundan
```

Population vs. present-only predictions

▶ Code

Predicted abundance when present (conditional) is always higher than population-scale abundance (response), since occurrence probability is below 100%. This is an implicit interaction **between the model's two parts**: tree cover's effect on population-scale abundance depends on its effect on both occurrence and abundance-when-present.

Simulating with an explicit interaction

Prey and competitors interact in their effect on abundance (when present); tree cover drives occurrence only. Predictors are simulated through the model's own inverse link (`exp()`, `plogis()`) so they stay in a valid range:

```
more_plots <- 4000
prey_abun <- dist_poisson(lambda = 6)
competitor_abun <- dist_poisson(lambda = 3)

pop_data <- tibble(
  tree_cov = generate(tree_cover, more_plots),
  prey = generate(prey_abun, more_plots),
  competitors = generate(competitor_abun, more_plots)
) |>
  unnest(everything()) |>
  mutate(
    abundance_pres = rpois(more_plots,
      lambda = exp(1 + 0.12 * prey - 0.15 * competitors - 0.045 * p
    occurence = rbinom(more_plots, size = 1, prob = plogis(-2 + 0.4
```

Fitting the hurdle model

- Code

`zi = ~ scl_tree_cov`: the zero-inflation (occurrence) part. The conditional formula's `scl_prey * scl_competitors` is an **explicit** interaction, layered on top of the log link's *implicit* one.

The interaction on the count scale

▶ Code

The interaction on the link scale

▶ Code

Unlike the count-scale slide, this one is linear in competitors by construction — the model's formula (`scl_prey * scl_competitors`) is fit additively on the log scale, so the **explicit** interaction shows up here as a straight line, with no additional curvature from the log link itself.

Isolating the interaction: rate ratios

To isolate something closer to “the interaction itself,” we move to the rate-ratio scale: for each one-SD increase in competitors, by what *factor* does the effect of one extra SD of prey get multiplied? That’s `exp()` of the interaction coefficient, and because it lives on the model’s own additive (log) scale, it doesn’t depend on where competitors are evaluated — unlike the count-scale slope.

▶ Code

Prey’s rate ratio drops from >1 (increasing abundance) at low competitor density to <1 (decreasing abundance) at high competitor density.

Predicted abundance, by competitor level

▶ Code

At low competitor density, abundance climbs steeply with prey; by the time competitors are abundant, the prey curve is nearly flat — the same shrinking slope we saw before, now shown as predicted trends rather than slopes.

Population-scale picture: low vs. high tree cover

▶ Code

Within each panel, the prey × competitors shape is the same. But the high-tree-cover panel sits above the low-tree-cover panel overall — more plots are occupied there. Tree cover mostly shifts *how many* plots have the species at all; prey and competitors jointly shape abundance within occupied plots.

A symmetric interaction, viewed from the other predictor

▶ Code

A two-way interaction is mathematically symmetric: `scl_prey:scl_competitors` is the same coefficient as `scl_competitors:scl_prey`. This view makes a different point than the prey-on-x-axis plot: regardless of prey availability, enough competitors drive abundance down to roughly the same low floor. Neither plot is “more correct” — reporting only one risks showing half the story (Spake et al. 2023).

Average predictions: one curve per predictor

▶ Code

`datagrid(..., grid_type = "counterfactual")` pairs each focal value with every observed row, then averages — a true average over the sample rather than a slope evaluated at the other predictors' means. Averaged over all observed competitor density, prey still shows a positive effect; averaged over all observed prey, competitors show a diminishing shape.

Population-scale average, by tree cover

▶ Code

The shapes barely change between tree-cover panels — the prey × competitors interaction lives entirely in the abundance-when-present part. What changes is the overall height: high-tree-cover populations sit consistently above low-tree-cover ones, a pure occupancy effect, cleanly separated from the abundance-when-present shape.

A full rate-ratio table, by model component

▶ Code

Prey's rate ratio crosses 1 somewhere around average competitor density (why its raw `summary()` estimate looked like noise); competitors suppress abundance more as prey increases; the interaction row (~0.81) is the single number behind both patterns, and it's the *same* number whichever predictor you treat as the moderator — the symmetry point from a few slides back, visible directly in the table. Tree cover's odds ratio in the zero-inflation part tells a simpler, interaction-free story about occupancy alone.

Part 3 takeaways

1. Non-linear link functions (logit, log) create an **implicit interaction**: every predictor's response-scale effect depends on the values of all other predictors, even with no interaction term in the formula
2. Raw GLM coefficients live on the link scale; `marginaleffects::slopes()` converts them to the natural (probability, count) scale for you
3. Multiplicative summaries (rate ratios, odds ratios) stay constant across the range of other predictors; additive, response-scale summaries do not
4. Whether an interaction "exists" can depend on the scale you're measuring on (Spake et al. 2023) — this applies just as much to *implicit* link-function interactions as to explicit interaction terms

Exercise 6 / Homework

Now apply what we learned today to your data!

Wrap-up and further reading

- Full write-ups with additional detail and 3D visualizations: Part 1, Part 2, Part 3
- `marginaleffects` documentation: https://marginaleffects.com/
- Spake et al. (2023), on interactions in GLMs: https://onlinelibrary.wiley.com/doi/full/10.1111/brv.12939
- Mize (2019), best practices for interaction effects: https://sociologicalscience.com/articles-v6-4-81/